CASO DE ESTUDIO 2 / ENTREGABLE 04

Integración de Machine Learning

Sistema de recompensas dinámico de MiMcDonald's

Segmentación, incentivos y trazabilidad

Proponemos integrar K-means en el pipeline para orientar los incentivos por segmento y producto. La POC implementa la segmentación y el cálculo de puntos; el piloto comercial, la gestión de modelos y la adopción de DVC se presentan como propuestas de evolución.

Integrante	Carné
Nelson García Bravatti	22434
Ricardo Chuy	221007
Diego Valenzuela	22309
Daniel Dubón	22233
Sergio Orellana	221122
Joaquín Puente	22296

Contenido

01 Objetivo y alcance

02 Segmentación e integración de reglas

03 Resultados e indicadores de evaluación

04 Evolución, DVC y CRISP-DM

05 Referencias y repositorio

Repositorio de GitHub

Machine Learning Engineering · Universidad del Valle de Guatemala

1. Objetivo y alcance

Objetivo general: proponer un sistema de puntos dinámico que ajuste los puntos otorgados según el producto y el comportamiento del cliente, y verificar su funcionamiento mediante un pipeline de datos sintéticos.

Objetivos específicos:

1. Inferir la arquitectura tecnológica actual del programa MiMcDonald's.
2. Diseñar un modelo de datos y una arquitectura Medallion que soporten puntos dinámicos.
3. Construir una prueba de concepto local con datos sintéticos.
4. Integrar un modelo de segmentación de clientes que, junto con el análisis de ventas, determine qué productos incentivar y para qué clientes.

Alcance: el proyecto se concentra en el **pipeline de datos** y en una comparación contable entre políticas de puntos. La aplicación, los sistemas POS y la API quedan fuera del alcance y se simulan mediante archivos CSV sintéticos. Por simplicidad, la POC se ejecuta en un entorno local; la misma estructura de capas puede trasladarse a una plataforma como Databricks en una implementación productiva.

La simulación usa saldo inicial cero, catálogo fijo y acreditación al terminar cada compra. No implementa topes diarios, vencimientos ni devoluciones. Conserva las mismas compras y canjes al comparar políticas, sin simular una respuesta del cliente a los incentivos. Trabajamos con clientes que tienen historial en la ventana de análisis y categorías que registran ventas. El escenario permite calcular los indicadores sin imputar valores para clientes nuevos o categorías sin compras.

Planteamos una arquitectura Medallion que conecta la preparación de datos, la segmentación y el cálculo de puntos. Utilizamos la prueba de concepto para evaluar la coherencia de las transacciones y explicar la asignación de incentivos.

2. Parámetros de la POC

Parámetro	Valor	Uso
Puntos por quetzal	10	Regla base del programa actual
Redondeo	Entero superior, por línea	Convención de la POC; la fuente pública no precisa el desglose por línea
Umbral de rezago	`indice_ventas` < 0.5	Un producto vende menos de la mitad del promedio de su categoría
Umbral de afinidad	`indice_afinidad` > 1.2	Un segmento compra la categoría más de 20 % por encima del promedio
Tope de multiplicador	x3	Máximo multiplicador combinado por línea
Número de segmentos	4	Grupos del modelo K-means
Ventana de análisis	Meses 1 y 2	Datos para perfiles, segmentos, ventas, afinidad y reglas
Ventana de aplicación	Mes 3	Compras a las que se aplican las reglas dinámicas
Fecha de referencia	Último día del mes 2	Base para calcular la recencia

Reglas de aplicación:

- Si varias reglas de tipo `producto` aplican a una línea, se usa la de mayor multiplicador; en empates, la de menor ID.
- Cada cliente recibe como máximo una regla de tipo `cliente`, la de su segmento.
- El multiplicador final es el producto de ambas reglas, sin superar el tope.
- Las líneas pagadas con puntos no generan acumulación. En cada compra se descuentan todos los canjes antes de acreditar los puntos ganados; el saldo inicial del período es cero.

3. Integración de ML en el pipeline

3.1 Modelo de segmentación

Aplicamos **K-means** (scikit-learn) a la tabla `perfil_clientes` para agrupar clientes con hábitos similares. Utilizamos las siguientes variables:

- recencia (días desde la última compra);
- frecuencia (compras por mes);
- gasto promedio;
- franja preferida.

Antes de entrenar, la franja preferida se convierte en columnas numéricas (una por franja), porque K-means solo trabaja con números. Se estandarizan las columnas y el bloque de franja se pondera por `1/√4 = 0.5`, para que sus cuatro columnas no dominen la distancia por su número.

K-means usa cuatro grupos, semilla `22434` y 20 inicializaciones. Los nombres se asignan mediante reglas reproducibles sobre los promedios: menor frecuencia → Ocasionales; mayor proporción de mañana entre los restantes → Madrugadores; mayor recencia entre los dos restantes → En riesgo; el último → Leales. La categoría preferida apoya la interpretación, sin entrar al entrenamiento:

Segmento	Comportamiento	Incentivo propio
Leales	Compran seguido y gastan bien	Ninguno adicional: ya compran seguido, así que solo reciben los multiplicadores de productos rezagados (sección 3.2)
Madrugadores	Compran casi solo en la mañana	Tipo `producto`: x2 en categorías de almuerzo (hamburguesas y pollo)
En riesgo	Presentan frecuencia intermedia y menor actividad reciente	Tipo `cliente`: x1.5 en cualquier compra
Ocasionales	Compran poco	Tipo `cliente`: x1.25 en cualquier compra

Obtenemos una silueta de **0.4280** con k = 4 y un ARI de **1.0000** frente a diez semillas alternativas. La silueta de k = 6 es mayor (**0.4499**): cuatro segmentos es una elección interpretativa de la POC, no el óptimo de esa métrica. El segmento En riesgo describe recencia y frecuencia; no es un predictor de abandono.

3.2 Asignación de multiplicadores

Además del incentivo propio de cada segmento, `reglas_puntos` incluye reglas para los productos rezagados, que se obtienen combinando dos análisis:

1. **Qué productos incentivar:** los marcados como `es_rezagado` en `ventas_productos`. Se comparan dentro de su categoría porque una ensalada no debe medirse contra una hamburguesa.
2. **A quién dirigir el incentivo:** los segmentos cuyo `indice_afinidad` con la categoría del producto supera el umbral. Esto dirige el multiplicador a clientes que ya eligen esa categoría con mayor frecuencia relativa; la hipótesis de que responderán mejor requiere evaluación en un piloto.

Este mecanismo aplica a cualquier segmento. Por ejemplo, si un postre vende poco y los clientes "Leales" compran postres más que el promedio, el x2 de ese postre se dirige a los Leales. Si ningún segmento muestra afinidad con la categoría, la regla se aplica a todos los clientes.

El flujo completo es: **Silver → perfil de clientes → segmentación → afinidad con productos → reglas de puntos → cálculo de puntos**. El modelo propone a quién incentivar y el cálculo aplica los topes, así cada punto se puede explicar.

3.3 Insights calculados

Insight	Pregunta que responde	Tabla de origen
Perfil de los segmentos	¿Qué tipos de clientes hay, cuántos son y cómo compran?	`perfil_clientes`
Productos rezagados	¿Qué productos necesitan impulso dentro de su categoría?	`ventas_productos`
Afinidad segmento-categoría	¿A qué clientes conviene dirigir cada incentivo?	`afinidad_segmento_categoria`
Emisión adicional del sistema dinámico	¿Cuántos puntos adicionales otorga frente a la regla fija, y en qué productos y segmentos se concentran?	`movimientos_puntos`

Estos insights responden **qué productos incentivar y para quién**. Si los incentivos realmente aumentan las ventas no se puede demostrar con datos sintéticos; eso requiere un piloto con clientes reales.

Resultados de la ejecución completa (semilla 22434):

Indicador	Resultado
Clientes por segmento	250 Leales; 249 Madrugadores; 255 En riesgo; 246 Ocasionales
Productos rezagados	4 de 28
Reglas generadas	8: seis de producto y dos de cliente
Movimientos de enero-marzo	49,525: 47,030 acumulaciones y 2,495 canjes
Emisión fija en marzo	4,080,306 puntos
Emisión dinámica en marzo	4,271,226 puntos
Emisión adicional en marzo	190,920 puntos, incremento del 4.68 %
Puntos canjeados en marzo	1,417,116 en ambas políticas
Clientes con puntos extra	605 de los 907 activos en marzo
Saldo final mínimo	150 puntos en ambas políticas

Los rezagados son Hamburguesa especial (índice 0.0635), Sándwich de pollo especial (0.0521), Pie de manzana (0.0343) y Ensalada especial (0.0462). Los primeros tres se incentivan para Leales y el último para En riesgo, según la afinidad de su categoría. Las reglas adicionales de Madrugadores, En riesgo y Ocasionales completan las ocho reglas.

Segmento	Puntos base en marzo	Puntos dinámicos	Puntos extra	Participación en puntos extra
Leales	3,024,031	3,053,660	29,629	15.52 %
Madrugadores	819,063	882,746	63,683	33.36 %
En riesgo	142,790	216,737	73,947	38.73 %
Ocasionales	94,422	118,083	23,661	12.39 %

En riesgo concentra la mayor emisión adicional, seguido de Madrugadores. Esto permite priorizar el control del presupuesto por segmento. Cuarto de libra aporta 24,243 puntos extra, el mayor total por producto: las reglas de cliente y de categoría también incentivan productos que no están rezagados. Por tanto, el costo del programa no debe atribuirse únicamente a los cuatro rezagados.

3.4 Propuestas de evolución

- **Escalabilidad:** proponemos migrar el pipeline a una plataforma como Databricks para trabajar con datos reales y a mayor escala.
- **Gestión de modelos:** proponemos registrar y versionar los modelos con MLflow.
- **Incentivos por horario:** planteamos aplicar multiplicadores a productos con pocas ventas en determinadas franjas del día.
- **Rentabilidad:** proponemos incorporar el margen de cada producto para priorizar los rezagados más rentables.
- **Prevención del abandono:** planteamos un modelo que estime el riesgo de que un cliente deje de comprar.
- **Evaluación comercial:** proponemos un piloto con un grupo de control que mantenga la regla fija para medir el impacto real.
- **Comunicación personalizada:** planteamos utilizar un LLM para redactar mensajes sobre los puntos extra.

3.5 Indicadores de éxito (en un piloto)

- **Frecuencia de visita:** si los clientes vuelven más seguido.
- **Ticket promedio:** si compran más por visita.
- **Ventas de los productos incentivados:** si los multiplicadores mueven los productos rezagados.
- **Costo en puntos del programa:** si el sistema sigue siendo rentable.

3.6 Investigación complementaria: DVC y trazabilidad

DVC (Data Version Control) es una herramienta de código abierto para relacionar versiones de datos y artefactos de Machine Learning con el código del proyecto. Complementa Git: este conserva archivos pequeños de metadatos (`.dvc`), mientras DVC administra el contenido en una caché local. `dvc add` registra rutas y hashes; un commit puede unir notebook y referencia al dataset exacto. Los datos administrados se excluyen de Git. Documentación de DVC.

Utilidad en la POC de MiMcDonald's

Artefacto	Información para versionar	Utilidad para el equipo
Bronze	CSV de clientes, productos, restaurantes, compras y líneas	Recuperar la simulación original o una variante con otra semilla o patrón de consumo
Silver y cuarentena	Datos aceptados y rechazos	Comparar cómo una versión de limpieza cambia la población analizada
Gold	Perfiles, ventas, afinidades, reglas y movimientos	Relacionar segmentación y emisión de puntos con las entradas y parámetros exactos
Modelo y métricas	Preprocesamiento, K-means, silueta y resultados	Comparar entrenamientos si se añade exportación del modelo; en el escenario actual permanece en memoria
Promociones futuras	Calendarios y condiciones comerciales	Explicar diferencias entre campañas; no existe todavía una fuente de promociones en esta POC

Como aplicación de DVC, planteamos conservar versiones de la simulación con distintas frecuencias de compra de clientes En riesgo. Esto permite comparar grupos, afinidades y puntos extra entre escenarios, y distinguir los efectos de un cambio de datos de los efectos de un cambio del algoritmo.

Reproducibilidad del pipeline

`dvc.yaml` declara etapas, comandos, dependencias, parámetros y salidas; `dvc.lock` registra el estado de una ejecución, incluidos hashes y valores de parámetros. Ambos se versionan con Git. `dvc repro` usa esas dependencias para ejecutar las etapas afectadas por cambios. Los comandos pueden ejecutar notebooks mediante `jupyter nbconvert --execute`; no es obligatorio convertirlos a scripts. Las copias ejecutadas deben escribirse aparte del notebook fuente para no modificar la propia dependencia. Estructura de dvc.yaml y dvc.lock y dvc repro.

La preparación de perfiles y la segmentación actualizan el mismo archivo `perfil_clientes.parquet`. Para integrar DVC, proponemos agrupar ambas operaciones en una etapa Gold/ML o asignarles rutas de salida distintas. Planteamos el flujo generación → limpieza → Gold/ML → puntos y la centralización de semilla, ventanas, k y umbrales en un archivo de parámetros.

DVC no instala el entorno ni garantiza resultados idénticos por sí solo. La reproducción también exige conservar versiones de Python y librerías, semillas, comandos y orden de ejecución. Registramos parte de esas versiones en las ejecuciones, pero `requirements.txt` no fija versiones exactas. La semilla hace deterministas los CSV con la misma configuración; no garantiza igualdad binaria de Parquet entre versiones de las herramientas.

Almacenamiento remoto y colaboración

Un remoto DVC guarda los archivos grandes; Git guarda código y metadatos. `dvc push` envía el contenido y `dvc pull` lo recupera. Puede usarse S3, Google Cloud Storage, Azure, Google Drive, SSH o una carpeta local/de red. Cada integrante necesita acceso al mismo almacenamiento; las credenciales particulares se configuran fuera de Git, por ejemplo con `--local`. DVC también ofrece versionamiento local sin un remoto; el remoto aporta respaldo y distribución.

Como el repositorio abarca todo el curso, proponemos inicializar DVC dentro de `Caso-Estudio-2` con `dvc init --subdir` para aislar su configuración.

Procedimiento de adopción propuesto:

1. Instalar DVC e inicializar el subproyecto. Migrar deliberadamente los datos que hoy están en Git: dejar de seguirlos con `git rm --cached` conservando los archivos locales. Esto no elimina copias de commits anteriores.
2. Para una primera estrategia de instantáneas, registrar `data/` con `dvc add data` y guardar `data.dvc`, las exclusiones y la configuración junto al código en Git.
3. Configurar un almacenamiento compartido con `dvc remote add -d`, cargarlo con `dvc push` y compartir el commit de Git. Los compañeros recuperan el código y ejecutan `dvc pull`.
4. Ante cambios de datos, actualizar la referencia con `dvc add data` y crear otro commit. Al volver a un commit anterior, `dvc checkout` restaura desde la caché; `dvc pull` recupera lo que falte del remoto.
5. Si se adopta el pipeline por etapas, sustituir el seguimiento global de `data/` por salidas separadas en `dvc.yaml`, evitando registrar dos veces las mismas rutas.

Criterio de adopción: mantenemos Git y la ejecución secuencial para el escenario actual, que utiliza datos pequeños y sintéticos. Planteamos DVC como una extensión de MLOps para conservar varias simulaciones, incorporar datos reales o compartir modelos. Su integración requiere configurar el seguimiento de datos y el almacenamiento. En una migración a Databricks, también proponemos definir qué versiones de tablas y artefactos conservar, ya que Delta, MLflow y DVC cubren necesidades complementarias de trazabilidad.

3.7 Relación con CRISP-DM

Fase	Evidencia en el proyecto
Comprensión del negocio	Tasa base, oportunidades, alcance y métricas del piloto (objetivos y métricas del piloto)
Comprensión de los datos	Modelo, diccionario y patrones sintéticos (generación de datos)
Preparación	Limpieza, cuarentena y agregados (notebooks 02-03)
Modelado	K-means, interpretación y afinidad (notebook 04)
Evaluación	Diagnóstico de grupos, pruebas de puntos y comparación de políticas (04-05)
Despliegue	Exportación local a Gold; integración productiva y piloto propuestos en 3.4-3.5

4. Referencias

- Databricks. (s. f.). *What is a medallion architecture?* https://www.databricks.com/glossary/medallion-architecture
- Hughes, A. M. (2012). *Strategic database marketing* (4.ª ed.). McGraw-Hill.
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.ª ed.). Wiley.
- McDonald's Guatemala. (s. f.). *Términos y condiciones del programa MiMcDonald's*. Consulta: 27 de septiembre de 2026.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 1981-1984.
- DVC. (s. f.). *Get started*, *Remote storage*, *dvc.yaml files*, *repro* e *init*. Consulta: 27 de septiembre de 2026.

5. Repositorio y reproducción

Repositorio de GitHub.

El proyecto incluye los cinco notebooks ejecutables, los datos de la simulación, las dependencias y las pruebas de integridad. La ejecución sigue el orden generación, limpieza, preparación de Gold, segmentación y cálculo de puntos.